

capitolo 6 – Introduzione all'analisi numerica

1. La qualità e la complessità degli algoritmi
2. I numeri macchina
3. Gli errori e l'attendibilità dei risultati
4. L'uso di Octave o MATLAB

6.1 La qualità e la complessità degli algoritmi

Indice

Un **algoritmo** è una procedura formata da **passi elementari non ambigui** che **risolve una classe di problemi** in un **tempo finito**.

Il **pensiero computazionale** analizza i problemi del mondo reale astraendone gli aspetti principali e scomponendoli in **sottoproblemi** più semplici, la cui soluzione si possa esprimere proprio in forma di **algoritmo**:

- per poterlo **eseguire con un computer**, un algoritmo va codificato mediante un **linguaggio di programmazione**;
- si esegue il programma per risolvere un'**istanza specifica** della classe generale di problemi, corrispondente ai particolari dati di input usati;
- il risultato dell'esecuzione, cioè l'insieme dei dati di output del programma, costituisce la **soluzione numerica** di quel particolare problema.

Il **calcolo scientifico** – o **calcolo numerico**, o **analisi numerica** – è la disciplina che si occupa della ricerca e dell'implementazione di **algoritmi** per risolvere con il computer problemi rappresentati in forma matematica.

6.1 La qualità e la complessità degli algoritmi

Indice

Un **algoritmo efficiente** usa in modo ottimale le risorse del computer.

L'**esecuzione di un algoritmo al computer richiede varie risorse**:

- lo spazio di memoria che serve per immagazzinare il programma e i dati
- la banda di comunicazione necessaria
- il numero delle porte logiche usate
- il tempo di calcolo impiegato per eseguire il programma.

Il **tempo di calcolo** all'atto pratico è di solito la risorsa più importante.

Per quantificare l'**efficienza** si deve stimare un tempo di calcolo «astratto», che rifletta le proprietà *intrinseche* dell'algoritmo, non della macchina su cui è eseguito. Faremo due ipotesi:

- durante l'esecuzione dell'algoritmo **ogni operazione elementare** – aritmetica, di assegnazione di valore o di confronto logico tra valori – **ha un costo pari a 1**;
- per l'algoritmo in esame si deve considerare il **caso peggiore**, vale a dire quello per cui i dati del problema richiedono il **tempo di esecuzione più lungo**.

6.1 La qualità e la complessità degli algoritmi

Indice

Un **algoritmo efficiente** usa in modo ottimale le risorse del computer.

Esempio: due algoritmi per cercare un valore in una sequenza non decrescente.

A. algoritmo di **ricerca lineare**:

- si inizia confrontando il valore cercato con il primo elemento del vettore;
- se i due valori non coincidono, si passa al confronto con il secondo elemento;
- e così via finché si trova il valore cercato, oppure si arriva all'ultimo elemento.

B. algoritmo di **ricerca binaria**:

- si confronta il valore cercato con l'elemento mediano del vettore: se coincidono, l'algoritmo termina;
- altrimenti, se il valore cercato è minore di quello trovato, si itera sulla prima metà del vettore; se è maggiore, si itera sulla seconda metà del vettore;
- si ripete poi la procedura, ogni volta dimezzando il vettore, finché è necessario.

caso peggiore per entrambi gli algoritmi: **il valore cercato è l'ultimo della sequenza, oppure non è presente nel vettore**

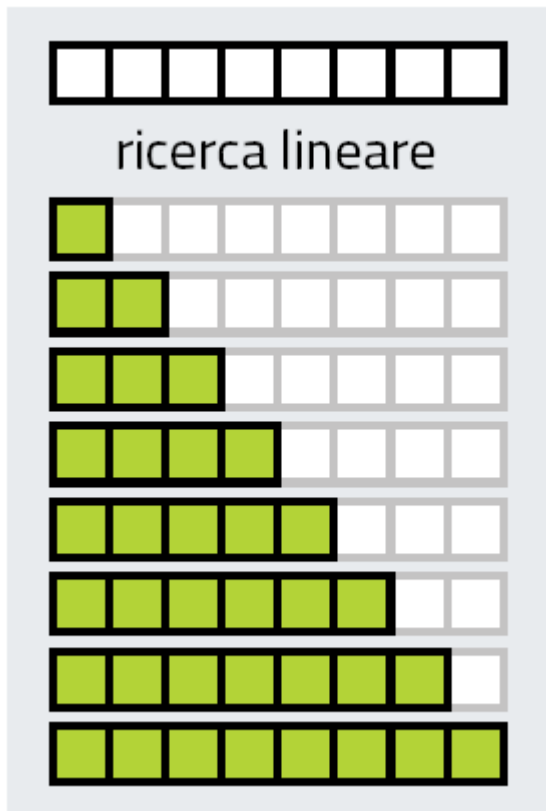
6.1 La qualità e la complessità degli algoritmi

Indice

Un **algoritmo efficiente** usa in modo ottimale le risorse del computer.

Esempio: due algoritmi per cercare un valore in una sequenza non decrescente.

A



B



la ricerca binaria è più efficiente rispetto a quella lineare: nel caso peggiore, infatti, risolve il problema con un numero minore di iterazioni

6.1 La qualità e la complessità degli algoritmi

Indice

Un **algoritmo efficiente** usa in modo ottimale le risorse del computer.

Esempio: due algoritmi per cercare un valore in una sequenza non decrescente.

Per n elementi la **ricerca lineare** richiede n operazioni, mentre la **ricerca binaria** ne richiede soltanto $\log_2 n$.

I **vantaggi di un algoritmo efficiente** diventano evidenti quando cresce la dimensione n del problema da risolvere:

n	$\log_2 n$	$n / \log_2 n$
10	~ 3	~ 3
100	~ 7	~ 14
1000	~ 10	~ 100
1 000 000	~ 20	$\sim 50\,000$

6.1 La qualità e la complessità degli algoritmi

Indice

La **complessità di un algoritmo** si descrive con la **notazione O-grande $O()$** .

Se **n** è la dimensione del problema da risolvere, **tra le parentesi di $O()$** si deve scrivere la **funzione di n** che rappresenta – nel caso peggiore – **il tempo di calcolo richiesto dall'algoritmo quando $n \rightarrow \infty$** .

$O()$ rappresenta la **complessità asintotica dell'algoritmo** considerato.

Per il problema della ricerca di un valore in un vettore crescente, dunque, l'algoritmo di **ricerca lineare** è **$O(n)$** , quello di **ricerca binaria** è **$O(\log n)$** .

Si può dimostrare che gli algoritmi più primitivi di **sorting** per il problema dell'**ordinamento dei valori di un vettore** hanno complessità **$O(n^2)$** .

Questo problema è così comune e importante che negli anni sono stati ideati e testati molti algoritmi diversi per risolverlo. I più efficienti scoperti finora, come il **merge sort**, hanno complessità **$O(n \cdot \log n)$** .

6.1 La qualità e la complessità degli algoritmi

Indice

Per i **problemi difficili** l'algoritmo risolutivo più efficiente ha come argomento di O-grande una **funzione che cresce rapidamente con n** .

L'**ordinamento dei valori** di un vettore è dunque un **problema più difficile** rispetto alla **ricerca di un valore** in un vettore crescente.

Infatti per il primo problema l'**algoritmo risolutivo più efficiente** ha complessità **$O(n \cdot \log n)$** , mentre per il secondo ha complessità **$O(\log n)$** .

6.1 La qualità e la complessità degli algoritmi

Per i **problemi difficili** l'algoritmo risolutivo più efficiente ha come argomento di O-grande una **funzione che cresce rapidamente con n** .

Un **problema molto difficile** è quello della **fattorizzazione di un numero**; qui la dimensione n del problema è data dal **numero delle cifre** del numero.

Per questo problema si è scoperto che **gli algoritmi migliori sono $O(2^{n/2})$** , cioè hanno un **tempo di calcolo che cresce in modo esponenziale con n** .

numero n di cifre	tempo di calcolo $T(n)$
4	1 μ s
20	256 μ s
40	262 ms
60	269 s = 4 minuti e mezzo
80	275 000 s = 76 ore
100	2.8×10^8 s = 9 anni

6.1 La qualità e la complessità degli algoritmi

Indice

Per i **problemi difficili** l'algoritmo risolutivo più efficiente ha come argomento di O-grande una **funzione che cresce rapidamente con n** .

I problemi in cui il **tempo di calcolo** per la soluzione aumenta al crescere di n come una **potenza di n** sono detti **di classe P**; la lettera **P** sta per ***polinomiale***.

I problemi come quello della **fattorizzazione** sono detti invece **di classe NP**, dove **NP** sta per ***non-deterministic polynomial***.

Non-deterministic significa che per eseguire l'algoritmo in tempo polinomiale servirebbe un numero sufficiente di computer che lavorano **in parallelo**.

6.1 La qualità e la complessità degli algoritmi

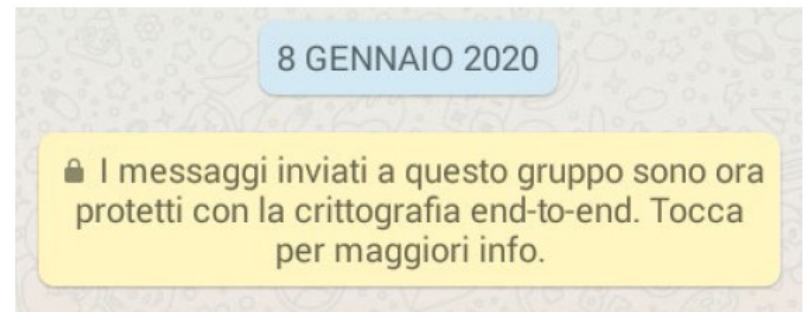
Indice

Per i **problemi difficili** l'algoritmo risolutivo più efficiente ha come argomento di O-grande una **funzione che cresce rapidamente con n** .

Il problema della **fattorizzazione** dunque è **molto difficile**;
ma è interessante notare che invece **il problema inverso è molto facile**.

Infatti, per sapere se il risultato dell'algoritmo di fattorizzazione è corretto, basta moltiplicare tra loro i fattori trovati e confrontare il risultato con il numero di partenza.

Queste due caratteristiche – l'estrema difficoltà del problema e l'estrema facilità del problema inverso – vengono sfruttate per crittografare i dati trasmessi in Rete con l'**algoritmo di cifratura RSA**.



6.2 I numeri macchina

Indice

La gamma di **numeri macchina** che si può rappresentare nel computer dipende dalla **lunghezza della parola**.

Nel computer i numeri sono rappresentati usando il **sistema binario**, cioè come una sequenza di **bit**, le cifre binarie che possono avere i valori **0 o 1**.

A ogni numero viene riservato uno **spazio di memoria** capace di contenere un particolare numero di bit, detto **parola**.

Se **L** è la lunghezza della parola, nel computer si possono rappresentare soltanto quei numeri la cui rappresentazione binaria consta di un numero di bit minore o uguale a **L** : questi sono chiamati **numeri macchina**.

6.2 I numeri macchina

Indice

La gamma di **numeri macchina** che si può rappresentare nel computer dipende dalla **lunghezza della parola**.

Numeri interi: con parole di lunghezza L si possono rappresentare tutti gli interi compresi nell'intervallo $[-2^{L-1}, +2^{L-1} - 1]$.

Esempio: una parola di **16 bit** permette di rappresentare tutti gli interi che appartengono all'intervallo **$[-32\ 768, 32\ 767]$** .

Altro esempio: con **32 bit** si possono rappresentare molti più interi, tutti quelli dell'intervallo **$[-2\ 147\ 483\ 648, 2\ 147\ 483\ 647]$** .

NB: per il computer **i numeri interi al di fuori dell'intervallo rappresentabile semplicemente non esistono**.

Esempio: se a un computer con parola di 16 bit si chiede di eseguire la somma **$32\ 767 + 1$** , il risultato sarà un **overflow**: il computer non può rappresentare internamente il risultato, che eccede lo spazio di memoria disponibile.

6.2 I numeri macchina

Indice

La gamma di **numeri macchina** che si può rappresentare nel computer dipende dalla **lunghezza della parola**.

Ogni computer quindi può trattare soltanto un **sottoinsieme finito** dell'insieme infinito **$\mathbb{Z}$** dei numeri relativi.

Gli interi del sottoinsieme, però, li rappresenta al proprio interno ***in modo esatto***.

Diversa è la situazione per i **numeri reali**: questi in generale hanno **un numero infinito di cifre decimali**, perciò nel computer li si può rappresentare soltanto ***in modo approssimato***.

6.2 I numeri macchina

Nel computer i **numeri macchina** che rappresentano i **numeri reali** sono espressi nel formato **floating point normalizzato in base 2**.

La rappresentazione generale dei **numeri floating point normalizzati** è:

$$x = (-1)^s \cdot (0.a_1a_2a_3 \dots a_N) \cdot \beta^E$$

- nel primo fattore, che rappresenta il **segno** del numero, **s** può valere 0 o 1;
- nel secondo fattore i simboli **a_i** rappresentano **le N cifre della mantissa**, che sono dette **cifre significative** del numero;
- nel terzo fattore **β** è la base di numerazione usata e l'esponente **E** è un intero dotato di segno.

NB: in generale per le cifre della mantissa vale la condizione $0 \leq a_i \leq \beta - 1$, ma per la **prima cifra significativa** si deve avere **$a_1 \neq 0$** .

Una **rappresentazione equivalente** alla precedente è:

$$x = (-1)^s \cdot m \cdot \beta^{E-N}$$

dove **m** è il numero naturale che rappresenta la mantissa.

6.2 I numeri macchina

Indice

Nel computer i **numeri macchina** che rappresentano i **numeri reali** sono espressi nel formato **floating point normalizzato in base 2**.

La rappresentazione generale dei **numeri floating point normalizzati** è:

$$x = (-1)^s \cdot (0.a_1a_2a_3 \dots a_N) \cdot \beta^E$$

Nel computer la base è $\beta = 2$, quindi l'unico valore possibile per la prima cifra della mantissa è $a_1 = 1$.

Perciò si dà questo valore per scontato e si memorizzano soltanto le cifre da a_2 in avanti, guadagnando così in precisione grazie a una cifra significativa in più.

NB: lo **zero** *non* appartiene all'insieme F dei numeri floating point normalizzati, perché avrebbe $a_1 = 0$. Nel computer perciò **il numero 0 è trattato a parte**.

6.2 I numeri macchina

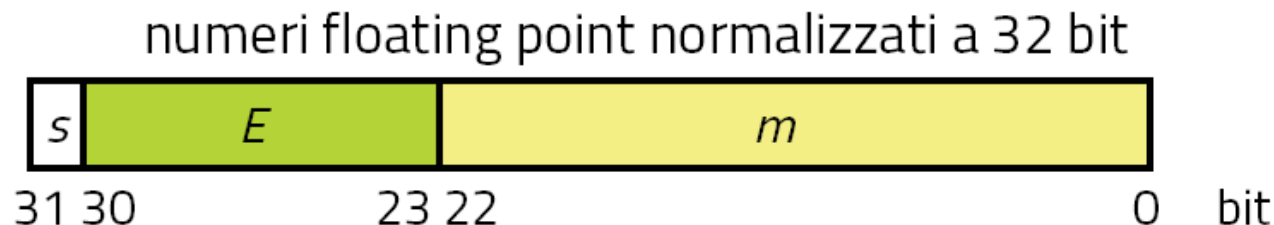
Indice

Nel computer i **numeri macchina** che rappresentano i **numeri reali** sono espressi nel formato **floating point normalizzato in base 2**.

La rappresentazione generale dei **numeri floating point normalizzati** è:

$$x = (-1)^s \cdot (0.a_1a_2a_3 \dots a_N) \cdot \beta^E$$

In **singola precisione** vengono riservati **4 byte** di memoria, cioè **32 bit**:
1 bit per il **segno** del numero, **8 bit** per l'**esponente** con il suo segno e
23 bit per la **mantissa** (che avrà quindi **$N = 24$** cifre significative).



La **singola precisione** permette di rappresentare numeri con valore assoluto compreso, in ordine di grandezza, tra **10^{-38}** e **10^{38}** , con **7 cifre significative**.

6.2 I numeri macchina

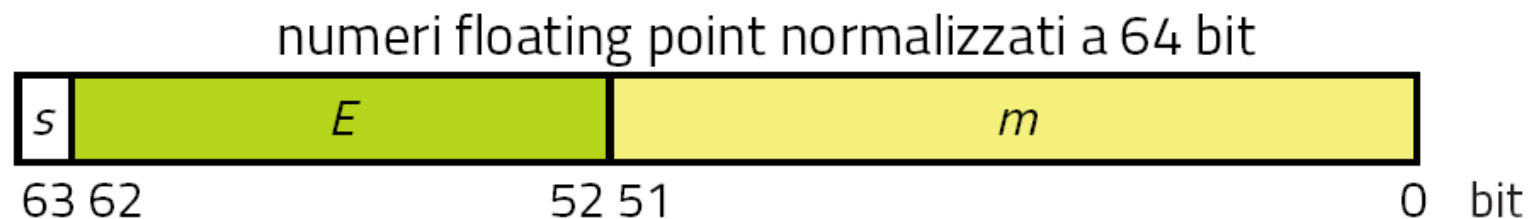
Indice

Nel computer i **numeri macchina** che rappresentano i **numeri reali** sono espressi nel formato **floating point normalizzato in base 2**.

La rappresentazione generale dei **numeri floating point normalizzati** è:

$$x = (-1)^s \cdot (0.a_1a_2a_3 \dots a_N) \cdot \beta^E$$

In **doppia precisione** vengono riservati **8 byte** di memoria, cioè **64 bit**:
1 bit per il **segno** del numero, **11 bit** per l'**esponente** con il suo segno e **52 bit** per la **mantissa** (che avrà quindi **$N = 53$** cifre significative).



La **doppia precisione** permette di rappresentare numeri con valore assoluto compreso, in ordine di grandezza, tra **10^{-308}** e **10^{308}** , con **16 cifre significative**.

L'**insieme dei numeri macchina**, che è **discreto** e **limitato**, rappresenta i numeri reali in modo **approssimato** tramite un **arrotondamento**.

La **rappresentazione dei numeri come parole di lunghezza fissa** ha numerose importanti conseguenze, tra cui queste:

- mentre l'**insieme R dei numeri reali è infinito**, in un **computer** se ne può rappresentare soltanto un **sottoinsieme finito F** , con limite inferiore e superiore;
- un **numero reale** non si rappresenta nel computer in modo esatto, ma con un **numero macchina** che lo approssima **arrotondando** per eccesso o per difetto;
- non sempre un'operazione tra numeri macchina produce un risultato: se il risultato è un numero più grande dell'estremo superiore di **F** , si ha un **overflow**; se invece il risultato è più piccolo dell'estremo inferiore di **F** , ha un **underflow**.

6.2 I numeri macchina

Indice

L'**insieme dei numeri macchina**, che è **discreto** e **limitato**, rappresenta i numeri reali in modo **approssimato** tramite un **arrotondamento**.

In algebra e in geometria siamo abituati a usare una **matematica del continuo** che è basata sul concetto di **numero reale**.

Quando usiamo il computer per fare calcoli numerici, invece, dobbiamo lavorare con una **matematica del discreto** basata sulla definizione dei **numeri macchina**.

ANALISI MATEMATICA

$\mathbb{R}$: numeri reali

insieme infinito

numeri di lunghezza infinita

matematica del continuo

CALCOLO NUMERICO

$\mathbb{F}$: numeri macchina

insieme finito

numeri di lunghezza finita

matematica del discreto

6.3 Gli errori e l'attendibilità dei risultati

Indice

L'**approssimazione** dei calcoli numerici fatti al computer si misura con l'**errore di arrotondamento** assoluto o relativo.

Ogni **calcolo numerico** è inevitabilmente **approssimato**, perché nel computer non si possono rappresentare in modo esatto i numeri reali.

Chiamiamo **$fl(x)$** il **numero macchina** usato per rappresentare il numero reale **x** .

Per descrivere l'**errore di arrotondamento** commesso si possono definire un **errore assoluto** e un **errore relativo**:

$$e_{\text{ASS}} = x - fl(x),$$

errore assoluto

$$e_{\text{REL}} = \frac{|x - fl(x)|}{|x|}$$

errore relativo

6.3 Gli errori e l'attendibilità dei risultati

Indice

L'**approssimazione** dei calcoli numerici fatti al computer si misura con l'**errore di arrotondamento** assoluto o relativo.

$$e_{\text{ASS}} = x - fl(x),$$

errore assoluto

$$e_{\text{REL}} = \frac{|x - fl(x)|}{|x|}$$

errore relativo

Esempio: gli errori di arrotondamento in singola e in doppia precisione per $\sqrt{2} = 1.414213562373095048801\dots$

singola precisione			doppia precisione		
$fl(\sqrt{2})$	e_{ASS}	e_{REL}	$fl(\sqrt{2})$	e_{ASS}	e_{REL}
1.4142136	-3.8×10^{-8}	2.7×10^{-8}	1.414213562373095	4.9×10^{-17}	3.5×10^{-17}

arrotondamento
per eccesso

arrotondamento
per difetto

6.3 Gli errori e l'attendibilità dei risultati

Indice

L'**approssimazione** dei calcoli numerici fatti al computer si misura con l'**errore di arrotondamento** assoluto o relativo.

$$e_{\text{ASS}} = x - fl(x),$$

errore assoluto

$$e_{\text{REL}} = \frac{|x - fl(x)|}{|x|}$$

errore relativo

Altro esempio: per un numero lontano da 1, come la **costante di Avogadro**

$$6.022140857 \times 10^{23}$$

in singola precisione si ha $e_{\text{ASS}} = -4.3 \times 10^{15}$, mentre $e_{\text{REL}} = 7.1 \times 10^{-9}$.

L'errore di arrotondamento assoluto non tiene conto dell'ordine di grandezza di x .

L'indicatore più utile è l'**errore relativo**, che misura l'**accuratezza del risultato**.

6.3 Gli errori e l'attendibilità dei risultati

Indice

La **epsilon macchina** è un indicatore della **precisione** del computer.

Per l'**errore relativo** vale la relazione:

$$e_{\text{REL}} = \frac{|x - fl(x)|}{|x|} \leq \frac{1}{2} \varepsilon_M$$

dove ε_M è detta **epsilon macchina** o **precisione macchina**;
rappresenta la distanza tra 1 e il più piccolo numero macchina maggiore di 1.

Nel computer (numeri floating point normalizzati in base 2) si ha che:

in singola precisione (24 cifre significative):	$\varepsilon_M = 2^{-23} = 1.19 \times 10^{-7}$
in doppia precisione (53 cifre significative):	$\varepsilon_M = 2^{-52} = 2.22 \times 10^{-16}$

L'errore relativo di arrotondamento quindi è molto piccolo. Ciò nonostante, anche quando si lavora in doppia precisione, la ripetizione e l'accumulo di errori piccoli in algoritmi lunghi e complessi può portare a conseguenze disastrose.

6.3 Gli errori e l'attendibilità dei risultati

Indice

La **epsilon macchina** è un indicatore della **precisione** del computer.

La **ripetizione e l'accumulo di errori piccoli** in algoritmi lunghi e complessi può portare a **conseguenze disastrose**.

Nel giugno 1996 il razzo **Ariane 5** dell'Agenzia Spaziale Europea ESA, durante il lancio inaugurale, deviò dalla giusta traiettoria e **finì per esplodere**.

Si scoprì poi che l'incidente era stato provocato dal software del computer di bordo: un **errore di arrotondamento** si era propagato nel corso dei calcoli, amplificandosi fino a causare un **overflow**.



6.3 Gli errori e l'attendibilità dei risultati

Indice

L'**aritmetica floating point** ha regole diverse da quelle dell'aritmetica esatta.

I risultati delle **operazioni aritmetiche tra numeri macchina** generalmente **non sono** numeri macchina.

Esempio: tutti quei casi in cui il risultato ha un numero di cifre decimali superiore alla precisione con cui il computer sta lavorando.

Bisogna perciò accontentarsi delle cosiddette **operazioni macchina**: a due numeri macchina viene associato un terzo numero macchina, che si ottiene **arrotondando** il risultato esatto dell'operazione aritmetica.

L'**aritmetica floating point** ha regole diverse da quelle dell'aritmetica esatta.

Non vale la regola dell'unicità dello zero, perché nel computer si può avere:

$$a + b = a \text{ anche con } b > 0$$

basta che **b** sia minore della distanza tra **a** e il numero floating point successivo.

Anche **la proprietà associativa e quella distributiva non valgono** in generale.

Esempio: se **a**, **b** e **c** sono tre numeri positivi con valore dell'ordine di 10^{308} , cioè vicino al massimo rappresentabile nel computer in doppia precisione, allora:

$a + (b - c)$ avrà un valore finito, ma **$(a + b) - c$** potrà dare **overflow**.

6.3 Gli errori e l'attendibilità dei risultati

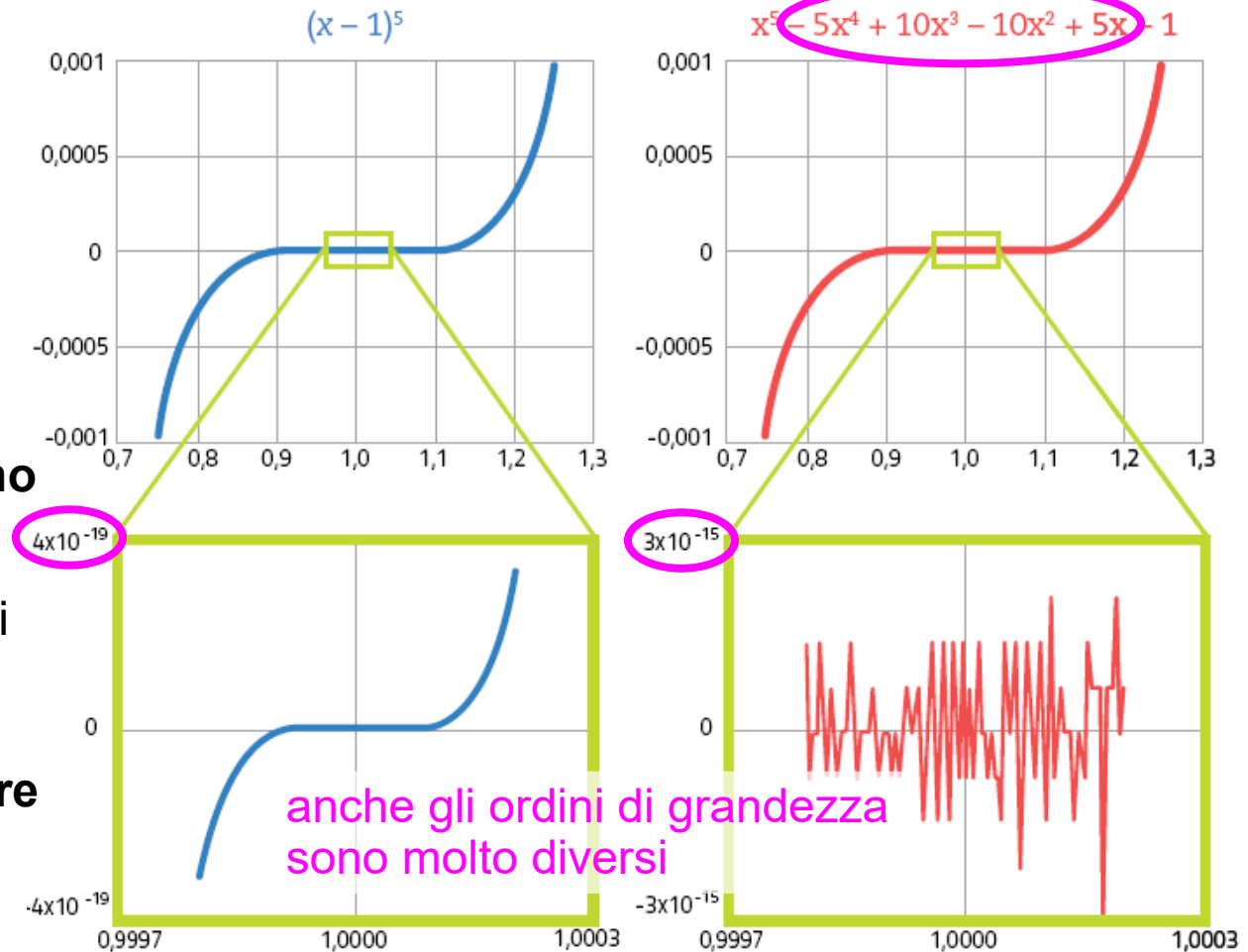
Indice

Quando si calcola la **differenza tra numeri quasi uguali**, si possono avere **errori di cancellazione numerica**.

Esempio: il calcolo numerico di due espressioni equivalenti nell'aritmetica esatta.

I grafici paiono identici ma in realtà **non lo sono** nei dintorni di $x = 1$.

Qui nel secondo caso si calcolano differenze tra termini quasi uguali: la risultante **perdita di cifre significative** genera oscillazioni caotiche.



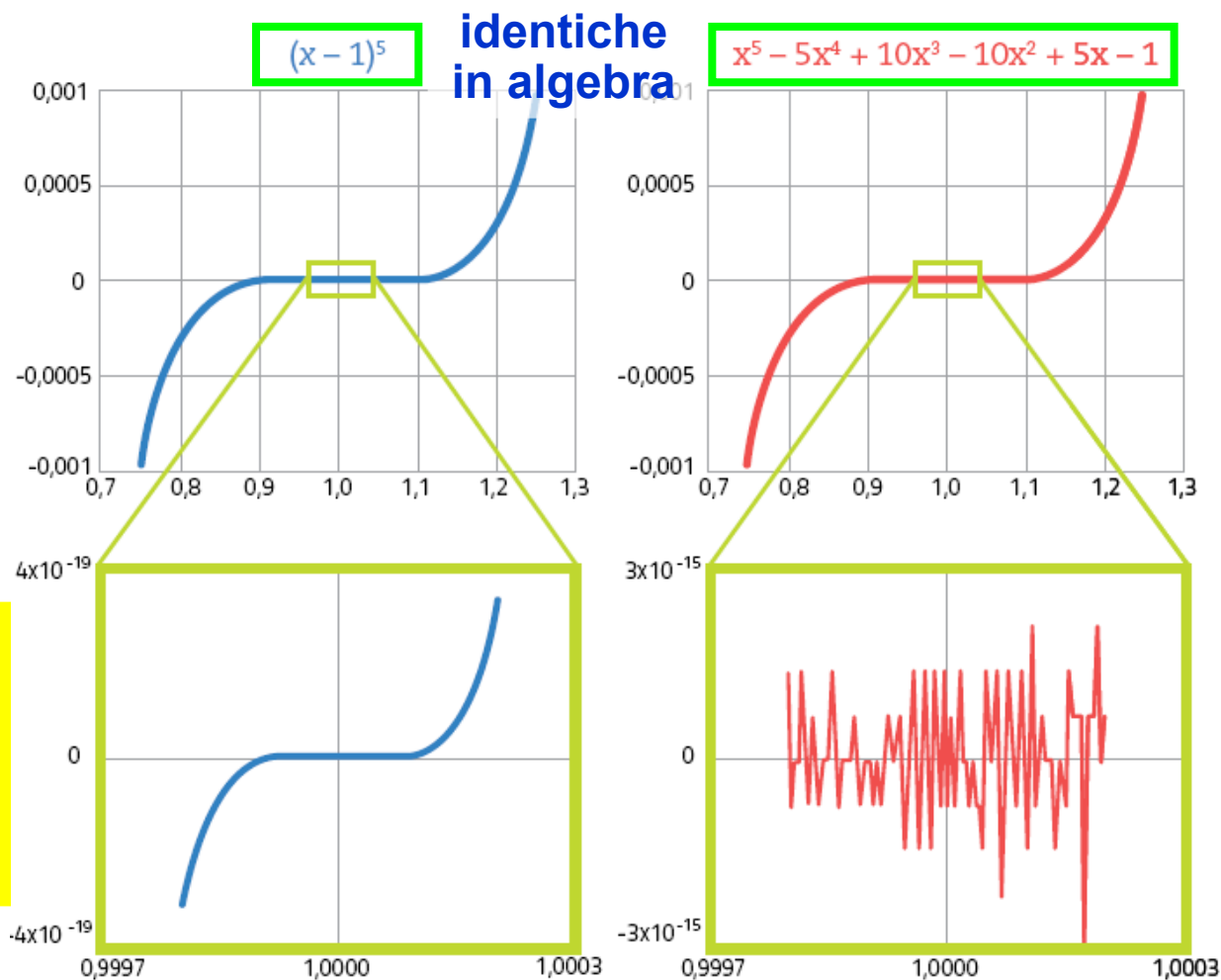
6.3 Gli errori e l'attendibilità dei risultati

Indice

Quando si calcola la **differenza tra numeri quasi uguali**, si possono avere **errori di cancellazione numerica**.

Esempio: il calcolo numerico di due espressioni equivalenti nell'aritmetica esatta.

Corollario importante: le identità algebriche non valgono in linea generale nell'aritmetica *floating point*.



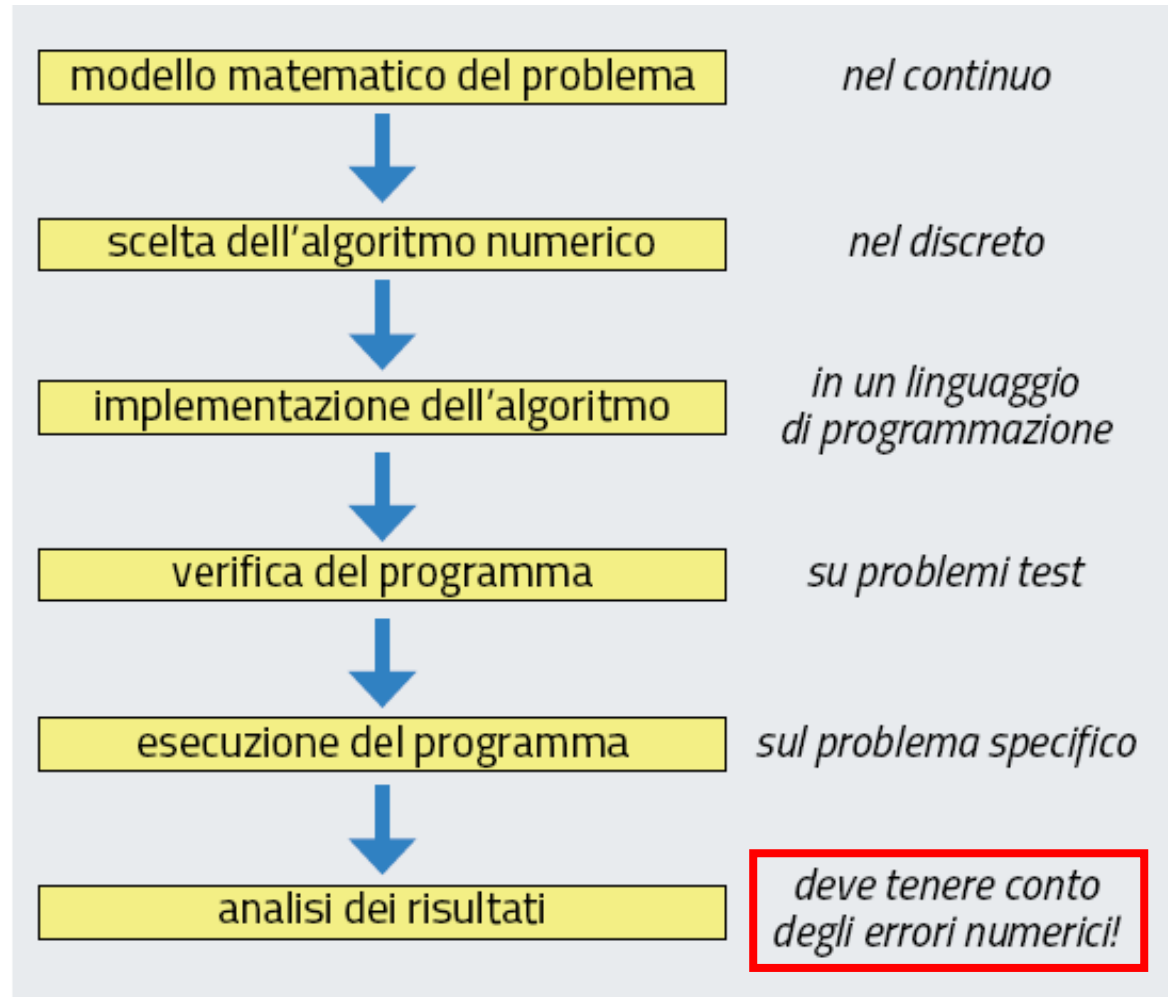
6.3 Gli errori e l'attendibilità dei risultati

Indice

Nel calcolo numerico non si mira a eliminare gli errori, che sono inevitabili, bensì a **controllare l'effetto degli errori sull'attendibilità dei risultati**.

Riassunto:

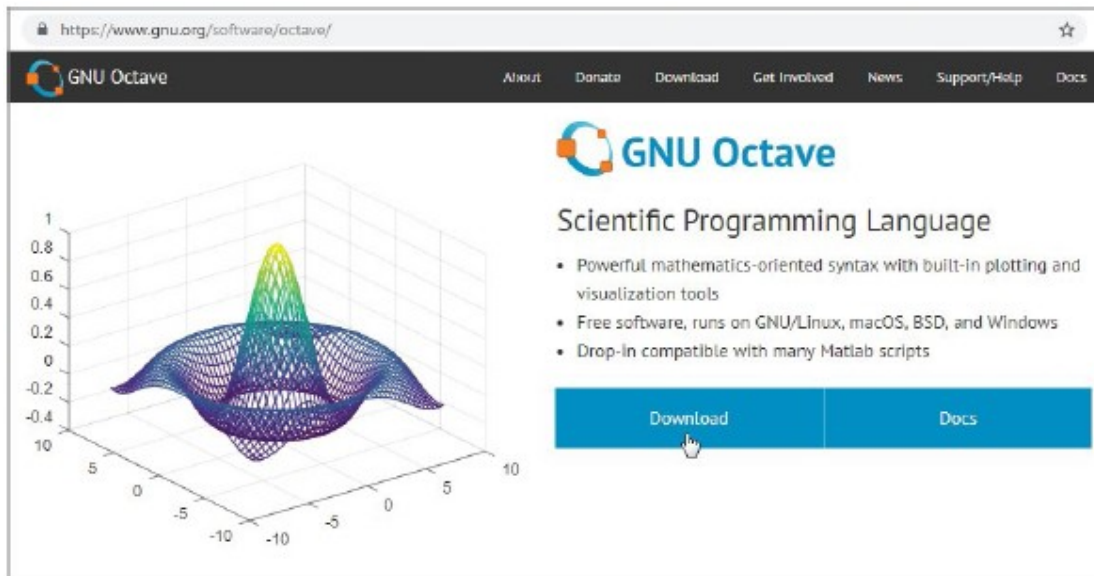
il procedimento per risolvere un problema scientifico mediante il calcolo numerico.



6.4 L'uso di Octave o MATLAB

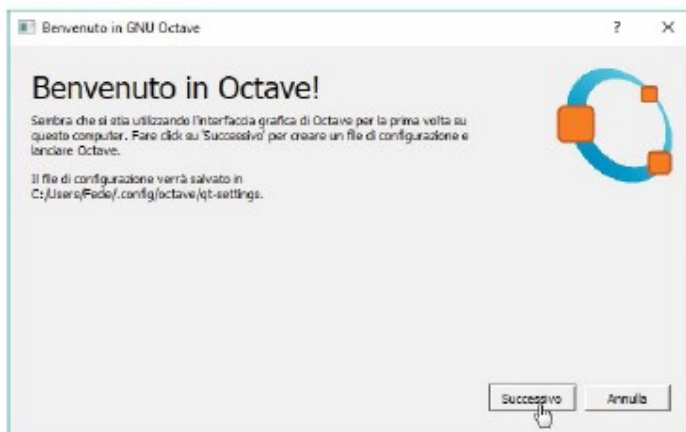
Indice

Octave è un freeware open source per il calcolo scientifico, simile a **MATLAB**.



Octave è un **PSE** (*problem-solving environment*), cioè un ambiente interattivo utile per **esercitarsi con il calcolo scientifico**.

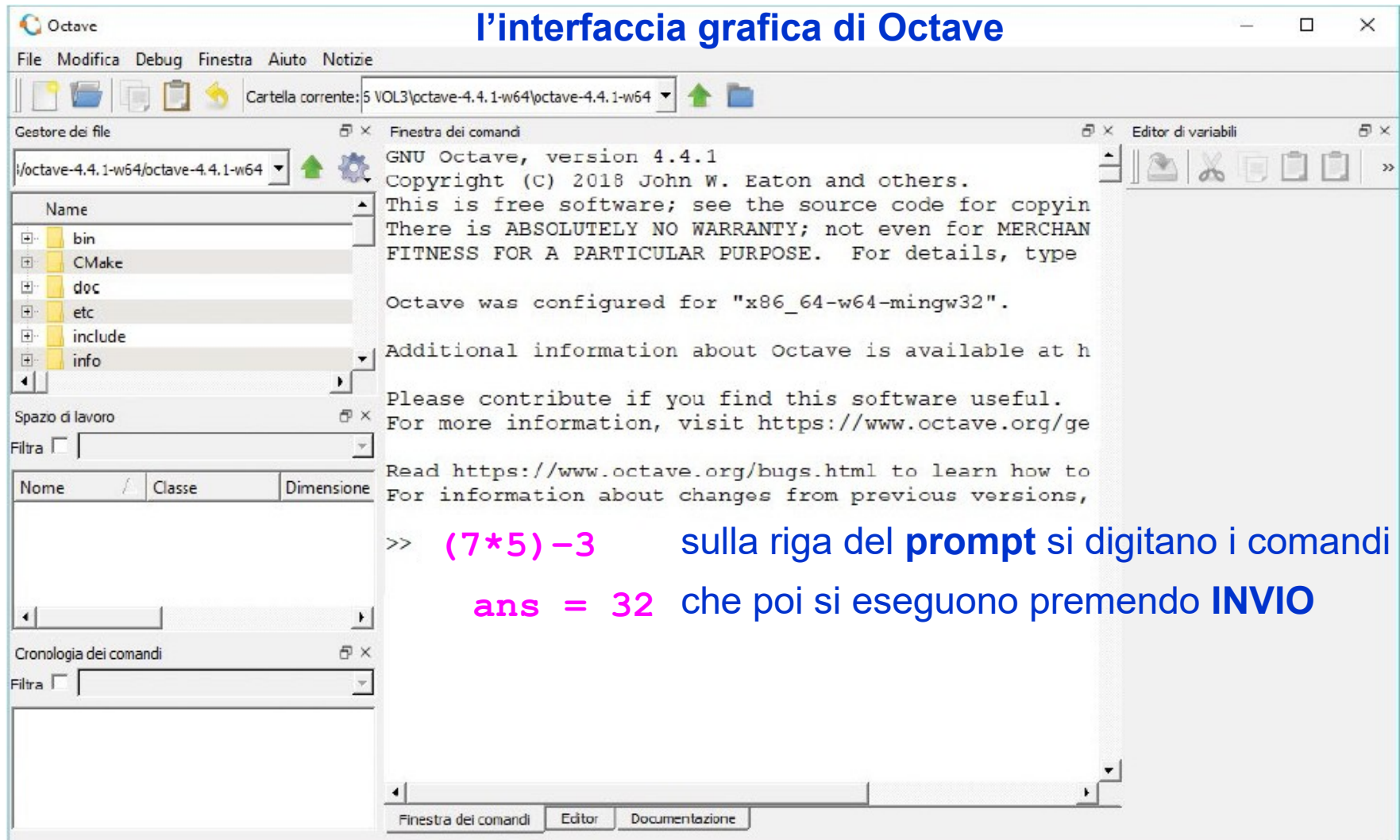
Lo si può scaricare dal sito **www.octave.org**.



6.4 L'uso di Octave o MATLAB

Indice

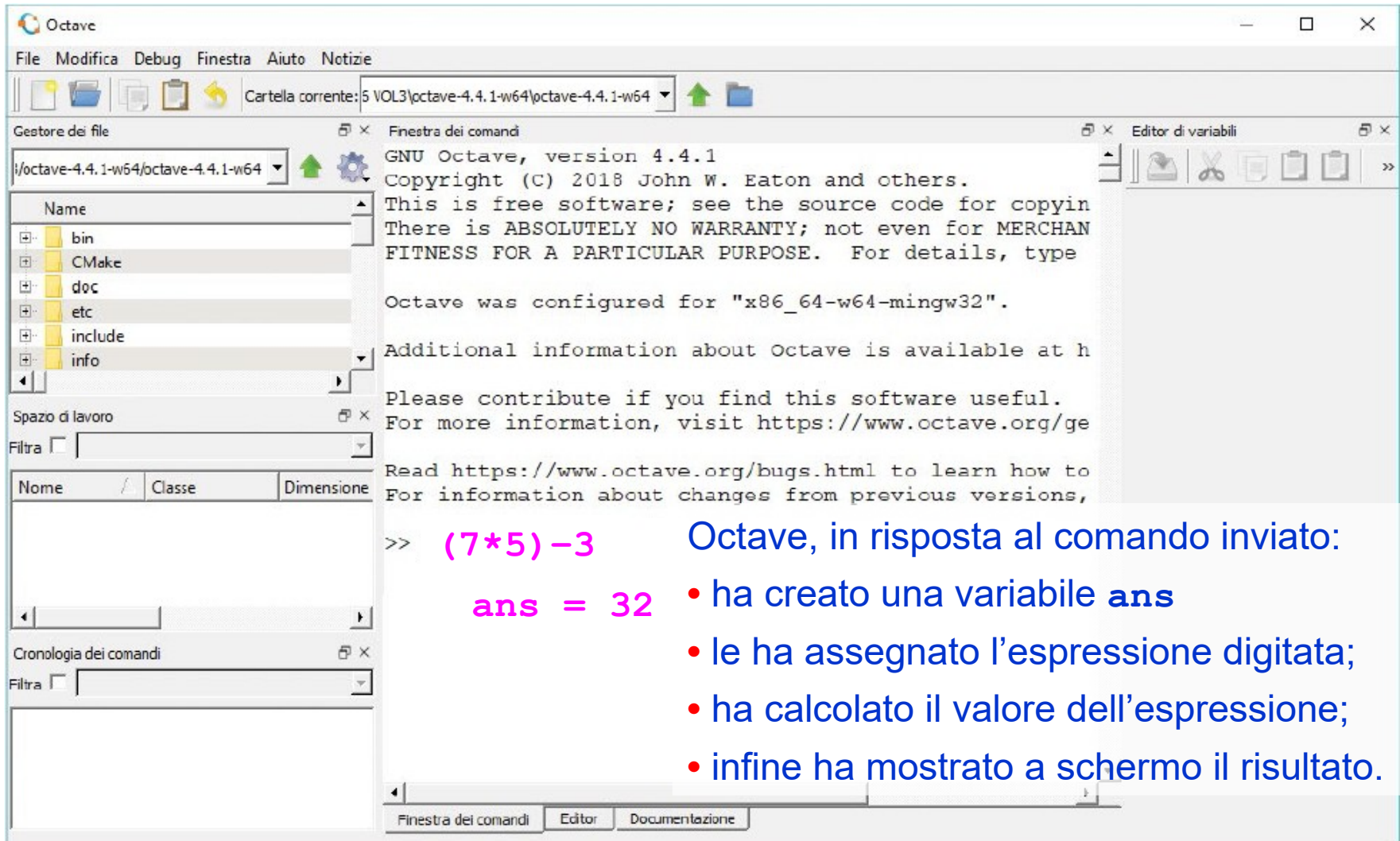
Octave è un freeware open source per il calcolo scientifico, simile a **MATLAB**.



6.4 L'uso di Octave o MATLAB

Indice

Octave è un freeware open source per il calcolo scientifico, simile a **MATLAB**.



The screenshot shows the GNU Octave 4.4.1 interface. The main window is titled "Octave" and contains a menu bar (File, Modifica, Debug, Finestra, Aiuto, Notizie) and a toolbar. The "Finestra dei comandi" (Command Window) is active, displaying the Octave startup message and the result of the command `(7*5)-3`. The result is `ans = 32`. The "Gestore dei file" (File Manager) is visible on the left, showing the directory structure of the Octave installation. The "Editor di variabili" (Variable Editor) is visible on the right, showing the variable `ans` with its value `32`.

GNU Octave, version 4.4.1
Copyright (C) 2018 John W. Eaton and others.
This is free software; see the source code for copying conditions. There is ABSOLUTELY NO WARRANTY; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. For details, type `info`.
Octave was configured for "x86_64-w64-mingw32".
Additional information about Octave is available at <https://www.octave.org>.
Please contribute if you find this software useful. For more information, visit <https://www.octave.org>.
Read <https://www.octave.org/bugs.html> to learn how to report bugs. For information about changes from previous versions, type `info`.

`>> (7*5)-3`
`ans = 32`

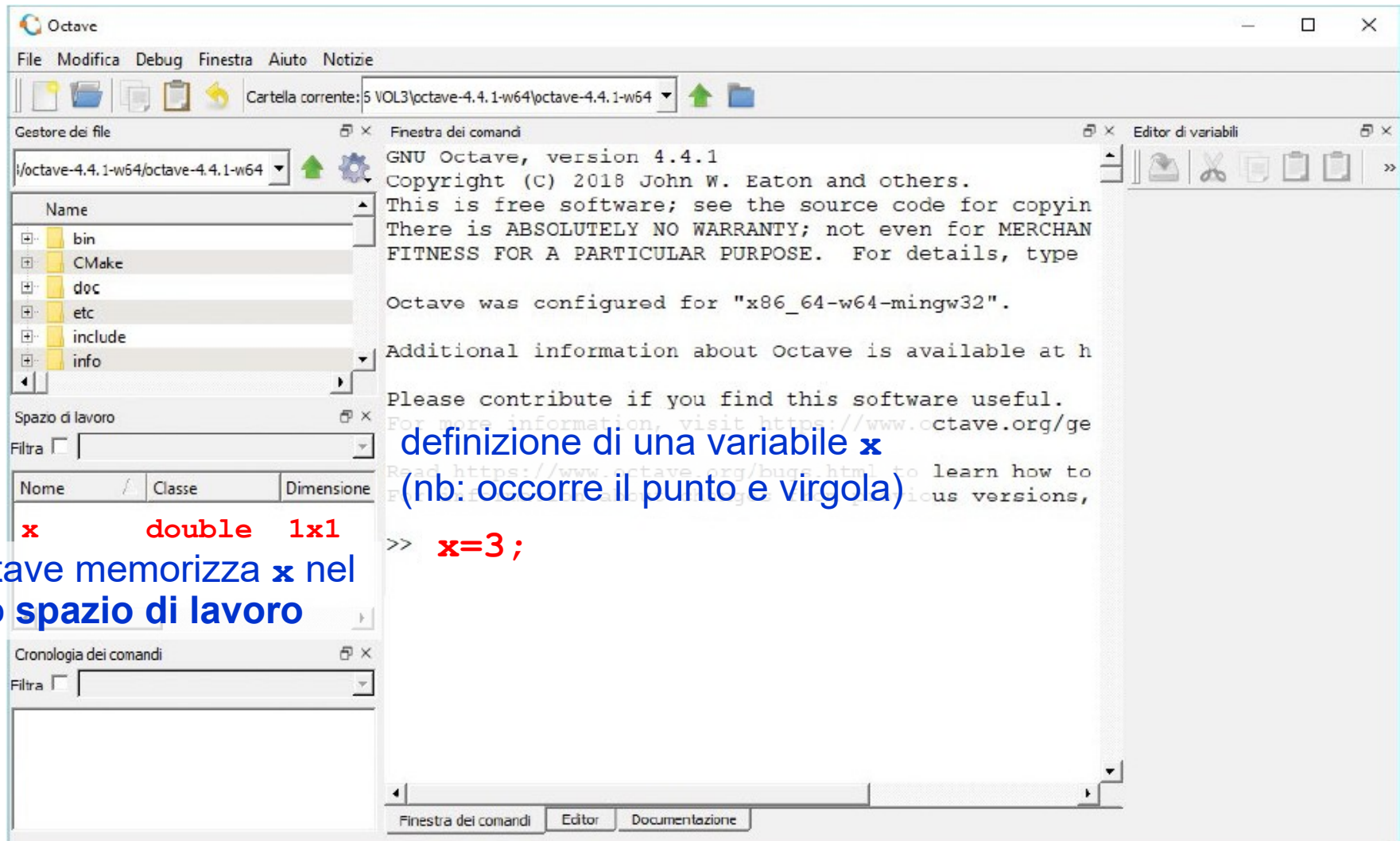
Octave, in risposta al comando inviato:

- ha creato una variabile `ans`
- le ha assegnato l'espressione digitata;
- ha calcolato il valore dell'espressione;
- infine ha mostrato a schermo il risultato.

6.4 L'uso di Octave o MATLAB

Indice

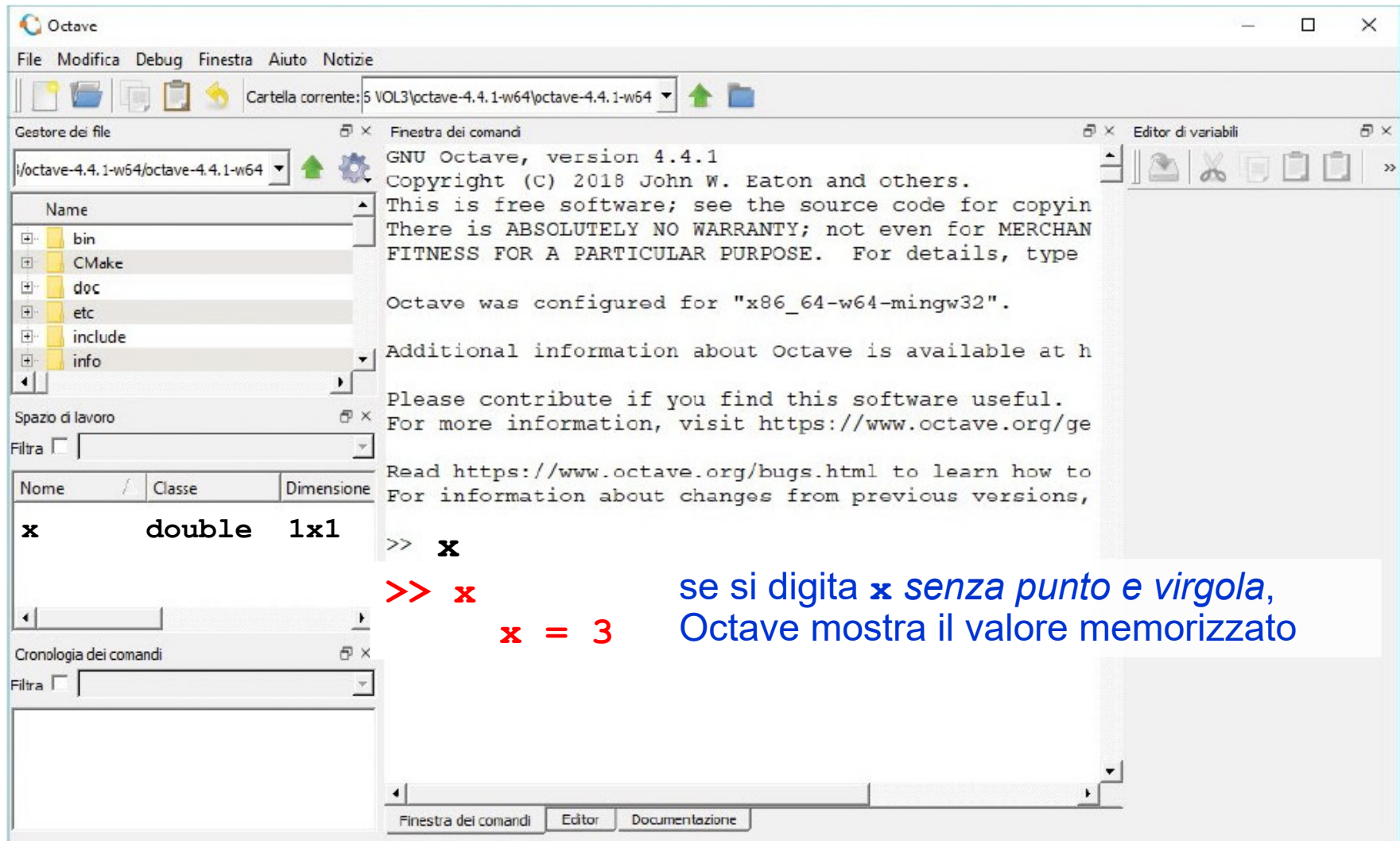
In **Octave** si possono **definire variabili** e usarle per **calcolare espressioni**.



6.4 L'uso di Octave o MATLAB

Indice

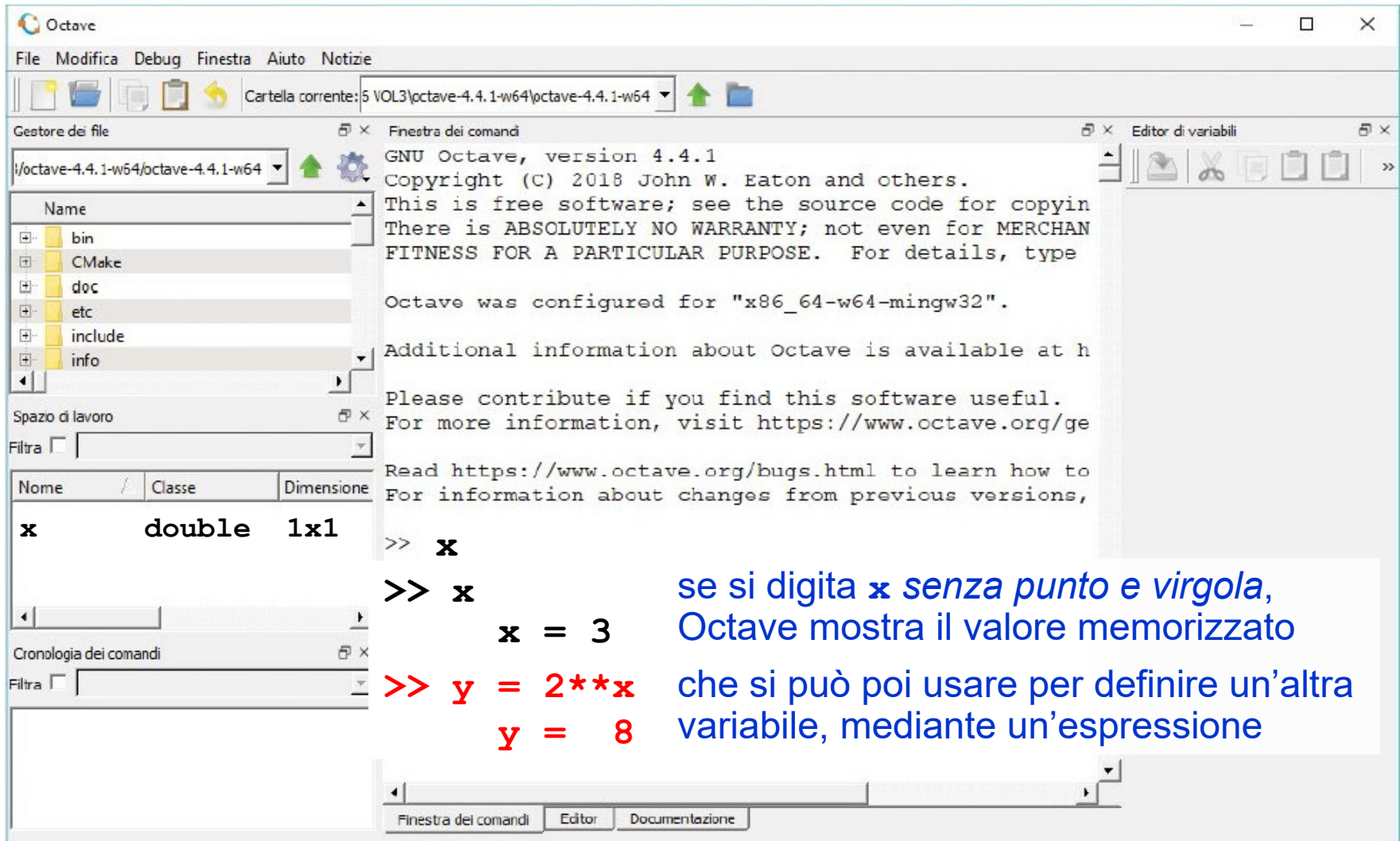
In **Octave** si possono **definire variabili** e usarle per **calcolare espressioni**.



6.4 L'uso di Octave o MATLAB

Indice

In **Octave** si possono **definire variabili** e usarle per **calcolare espressioni**.



The screenshot shows the GNU Octave 4.4.1 interface. The main window is titled "Octave" and contains a menu bar (File, Modifica, Debug, Finestra, Aiuto, Notizie) and a toolbar. The "Finestra dei comandi" (Command Window) is active, displaying the Octave startup message and the current directory path: `/octave-4.4.1-w64/octave-4.4.1-w64`. The "Editor di variabili" (Variable Editor) is also visible, showing a table of variables.

Nome	Classe	Dimensione
x	double	1x1

The command window shows the following commands and output:

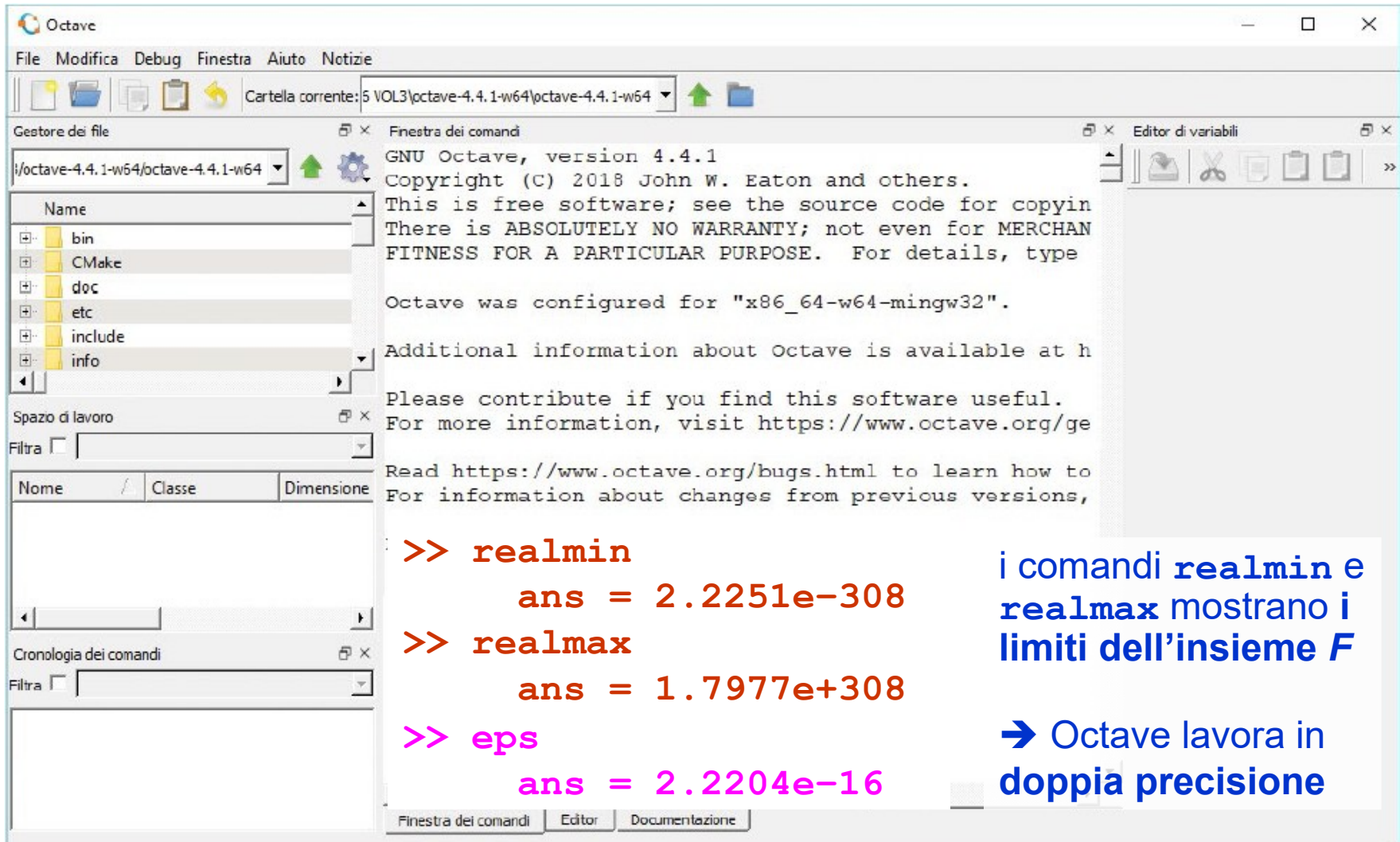
```
>> x
x = 3
>> y = 2**x
y = 8
```

se si digita **x** senza punto e virgola, Octave mostra il valore memorizzato che si può poi usare per definire un'altra variabile, mediante un'espressione

6.4 L'uso di Octave o MATLAB

Indice

Octave lavora in **doppia precisione**; l'epsilon macchina è dell'ordine di 10^{-16} .



The screenshot shows the GNU Octave 4.4.1 interface. The command window displays the following text:

```
GNU Octave, version 4.4.1
Copyright (C) 2018 John W. Eaton and others.
This is free software; see the source code for copyin
There is ABSOLUTELY NO WARRANTY; not even for MERCHAN
FITNESS FOR A PARTICULAR PURPOSE. For details, type

Octave was configured for "x86_64-w64-mingw32".

Additional information about Octave is available at h
Please contribute if you find this software useful.
For more information, visit https://www.octave.org/ge
Read https://www.octave.org/bugs.html to learn how to
For information about changes from previous versions,
```

Below this text, three commands are entered in the command window:

```
>> realmin
      ans = 2.2251e-308
>> realmax
      ans = 1.7977e+308
>> eps
      ans = 2.2204e-16
```

To the right of the command window, there is a blue text box with the following text:

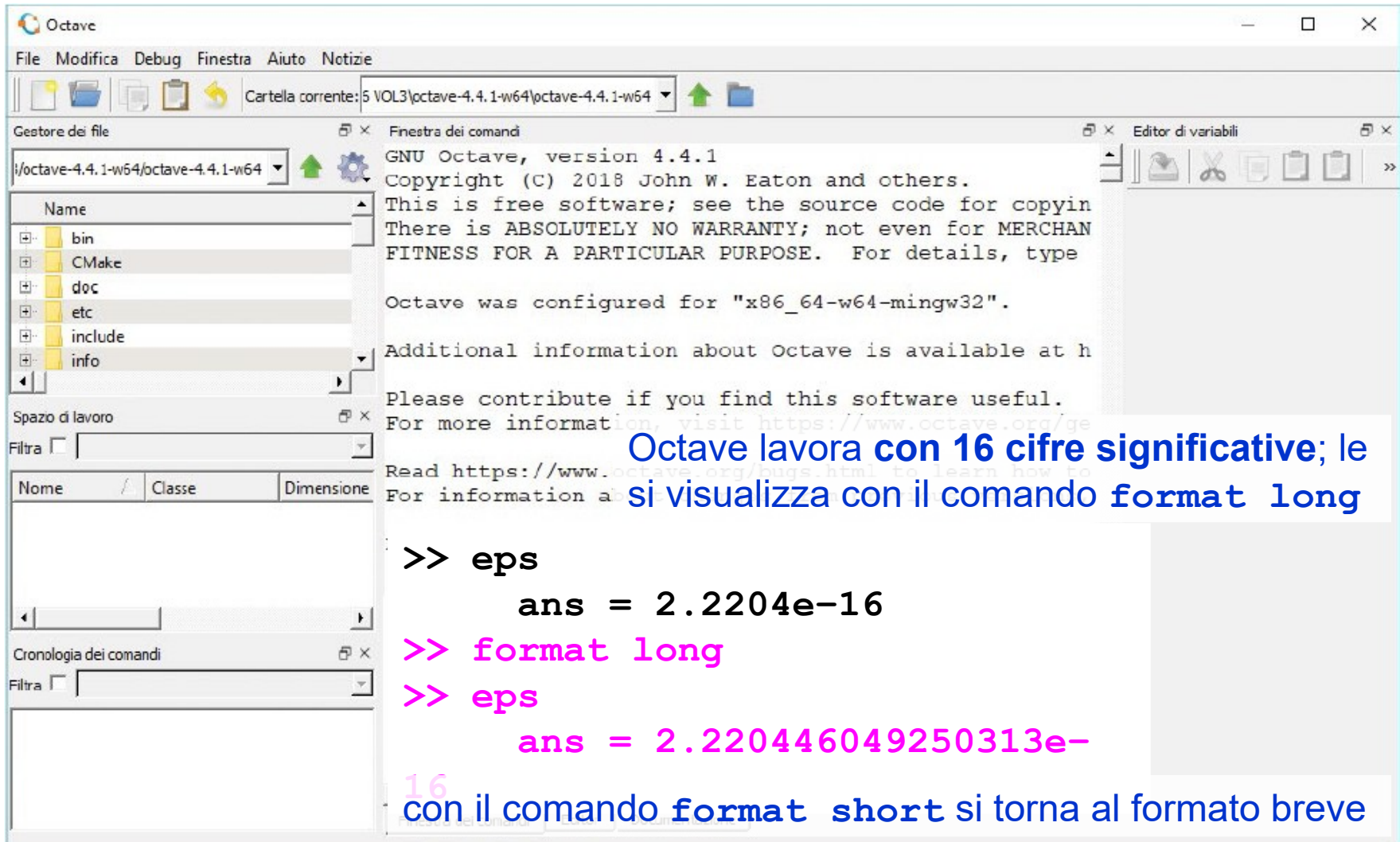
i comandi **realmin** e **realmax** mostrano i limiti dell'insieme F

→ Octave lavora in **doppia precisione**

6.4 L'uso di Octave o MATLAB

Indice

Octave lavora in **doppia precisione**; l'epsilon macchina è dell'ordine di 10^{-16} .



GNU Octave, version 4.4.1
Copyright (C) 2018 John W. Eaton and others.
This is free software; see the source code for copying
There is ABSOLUTELY NO WARRANTY; not even for MERCHANTABILITY
FITNESS FOR A PARTICULAR PURPOSE. For details, type
Octave was configured for "x86_64-w64-mingw32".
Additional information about Octave is available at <https://www.octave.org/>
Please contribute if you find this software useful.
For more information, visit <https://www.octave.org/>
Read <https://www.octave.org/bugs.html> to learn how to
For information about

**Octave lavora con 16 cifre significative; le
si visualizza con il comando `format long`**

```
>> eps  
ans = 2.2204e-16  
>> format long  
>> eps  
ans = 2.220446049250313e-16
```

con il comando `format short` si torna al formato breve

Octave fornisce molte **funzioni matematiche standard** di sistema.

Esempi:

- **abs ()** restituisce il **valore assoluto** dell'argomento
- **sqrt ()** restituisce la **radice quadrata** dell'argomento
- **sin ()**, **cos ()** e **tg ()** sono le funzioni trigonometriche **seno**, **coseno** e **tangente** per argomenti espressi in radianti
- **exp ()** restituisce l'**esponenziale**, cioè la potenza che ha come esponente l'argomento e come base il *numero di Eulero* (o di *Nepero*) $e = 2.71828...$
- **log ()**, **log10 ()** e **log2 ()** restituiscono i **logaritmi** in base e , 10 e 2

Si possono inoltre richiamare facilmente le principali costanti matematiche.

Esempio: scrivere **pi** in Octave equivale a scrivere 3.141592653589793.

L'**argomento** tra le parentesi di una funzione può essere un numero, un'espressione oppure una funzione, come dimostra questo esempio:

```
>> sqrt(abs(sin(pi/2)*log(e^-4)))  
ans = 2
```

In Octave una **funzione anonima** contiene un'unica istruzione eseguibile.

In Octave una funzione si può definire come ***anonymous function***; in tal caso la definizione deve consistere in un'unica istruzione.

La funzione si definisce mediante un puntatore chiamato ***function handle*** e indicato dal simbolo @, con la seguente sintassi:

```
>> nome_funzione = @(argomenti) definizione della funzione
```

dove gli argomenti, se sono più di uno, vanno separati da virgole.

In Octave una **funzione anonima** contiene un'unica istruzione eseguibile.

Per esempio, per definire la funzione $f_1(x) = x/(1+x^2)$ si può scrivere:

```
>> funz1 = @(x) x/(1+x^2)
```

mentre la funzione $f_2(x,y) = y/(1+x^2)$ si può definire scrivendo:

```
>> funz2 = @(x,y) y/(1+x^2)
```

Le funzioni anonime si possono poi richiamare semplicemente per valori, così:

```
>> funz1(2)  
ans = 0.40000
```

oppure tramite l'assegnazione a variabile, come in questo esempio:

```
>> z = funz2(3,5)  
z = 0.50000
```

6.4 L'uso di Octave o MATLAB

Indice

I **programmi** o **script** per fare calcoli numerici con Octave o MATLAB si salvano in **file di testo con estensione .m**.

In alternativa alle funzioni anonime, si possono definire **user-defined function** memorizzate in file esterni, che si richiameranno da Octave quando occorre.

Le funzioni definite dall'utente in file esterni, diversamente da quelle anonime, possono anche consistere in **script formati da numerose istruzioni**.

I **codici** di Octave sono formati da **testo non formattato**: li si può scrivere con un qualsiasi editor di testi, salvandoli poi come **file con estensione .m**.

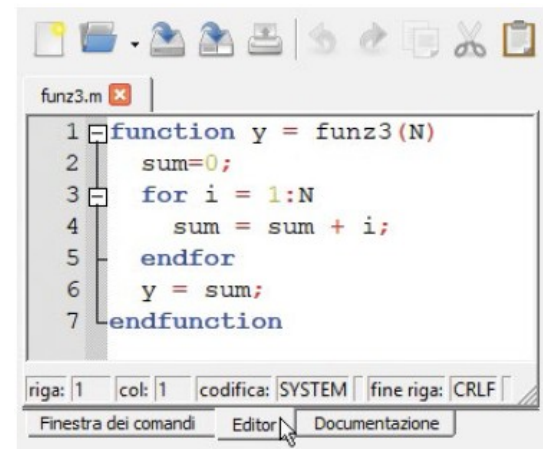
Octave fornisce anche un comodo **IDE**, che si apre facendo clic in basso sulla linguetta **Editor**. Qui si possono **scrivere, salvare ed eseguire codici**.

Il **comando chdir** indica a Octave dove trovare i file:

```
>> chdir('C:/octave')
```

```
>> funz3(100)
```

```
ans = 5050
```



Octave e MATLAB sono applicazioni progettate per il **calcolo matriciale**.

Un vettore-riga si definisce elencandone gli elementi tra parentesi quadre, separati da virgole o da semplici spazi bianchi. Per esempio:

```
>> v = [1, -3, 4]
v =
     1     -3      4
```

Un vettore-colonna si definisce allo stesso modo, ma usando come separatore il punto e virgola (oppure l'a capo). Per esempio:

```
>> vc = [1; -3; 4]
vc =
     1
    -3
     4
```

Si può definire un vettore-colonna anche come trasposizione di un vettore-riga, mediante l'operatore apice ' di Octave: nell'esempio qui sopra, basta scrivere `>> vc=v'`.

Octave e MATLAB sono applicazioni progettate per il **calcolo matriciale**.

Una matrice si assegna come elenco ordinato di numeri o espressioni in cui le virgole o gli spazi separano gli elementi di una stessa riga, mentre i punti e virgola o l'a capo separano le righe. Per esempio:

```
>> M=[1 0 -4 5; -2 7 6 0; 0 3 1 -5]
```

```
M =
```

1	0	-4	5
-2	7	6	0
0	3	1	-5

6.4 L'uso di Octave o MATLAB

Indice

Octave e MATLAB sono applicazioni progettate per il **calcolo matriciale**.

Uno strumento molto utile è la **colon notation**, che permette di rappresentare i vettori riga in modo conciso con la scorciatoia dell'operatore : (due punti).

min:p:max genera un vettore con primo elemento uguale a **min** e i successivi equispaziati, con passo **p**, fino al massimo valore che non supera **max**.

Esempio: creare un array con tutti i numeri dispari minori di 10

```
>> vd=[1:2:10]  
vd = 1 3 5 7 9
```

Il passo, se non viene specificato, è per default pari a 1.

Octave e MATLAB sono applicazioni progettate per il **calcolo matriciale**.

Un **ciclo iterativo** che inizia con l'istruzione `for i = 1:N` verrà ripetuto per tutti i valori tra gli estremi 1 e N inclusi, grazie alla definizione del vettore-contatore `i`.

Il comando `linspace(val_min, val_max, N)` genera invece un vettore con N elementi e con valori equispaziati tra gli estremi `val_min` e `val_max` (compresi).

Esempio: creare un vettore con 5 elementi equispaziati tra 0.5 e 1.5

```
>> z = linspace(0.5, 1.5, 5)
      z = 0.50000 0.75000 1.00000 1.25000 1.50000
```

Se il **numero di elementi** di un vettore non viene assegnato, sarà per default **100**.

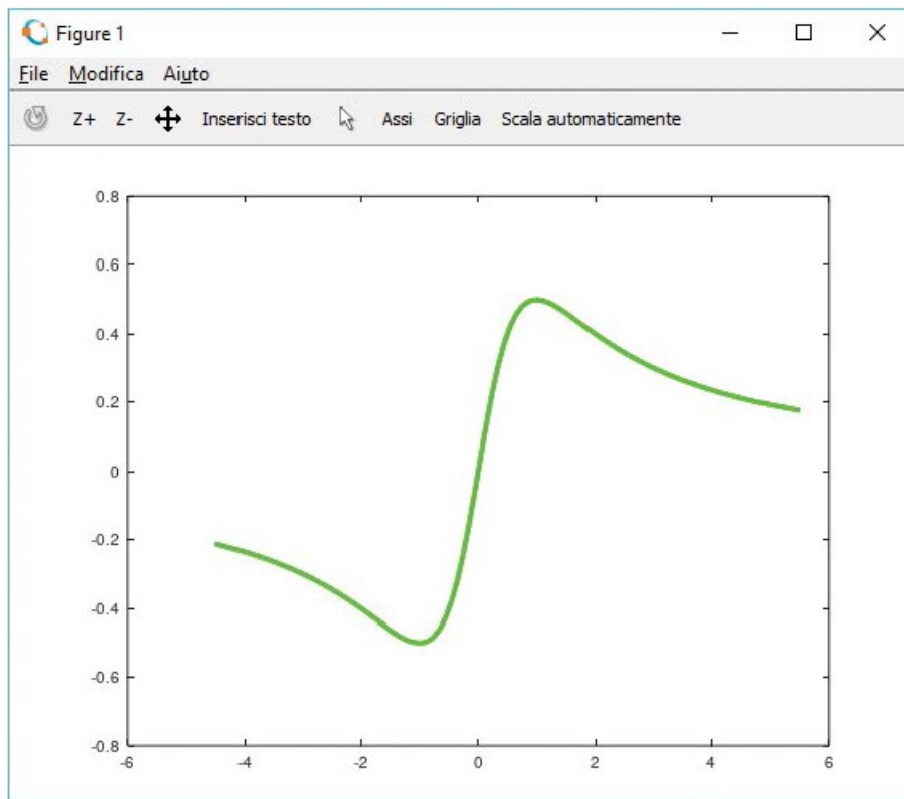
6.4 L'uso di Octave o MATLAB

Indice

In Octave si possono produrre **grafici** con i comandi **plot** oppure **fplot**.

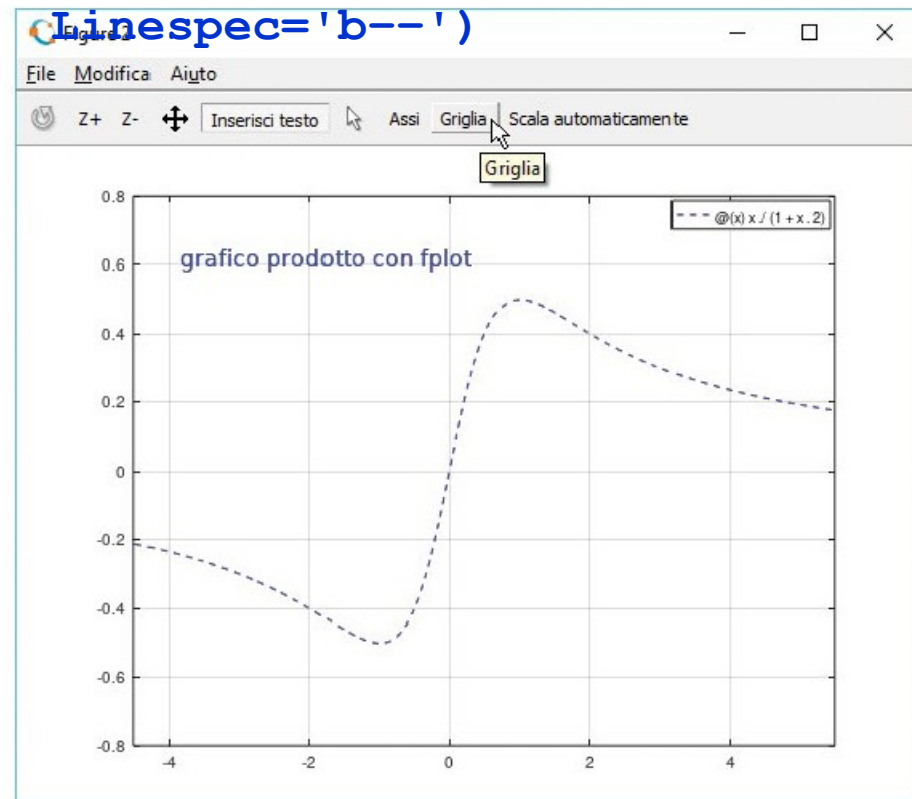
Esempio con **plot**:

```
>> x = linspace(-4.5,5.5);  
>> y = x./(1+x.^2);  
>> plot(x,y,'g','Linewidth',3)
```



Esempio con **fplot**:

```
>> funz_an = @(x)  
x./(1+x.^2);  
>> fplot(funz_an, [-4.5,5.5],  
Linespec='b--')
```



6.4 L'uso di Octave o MATLAB

Indice

In Octave si possono produrre **grafici** con i comandi **plot** oppure **fplot**.

Esempio: un grafico che mostra simultaneamente **più funzioni**

```
>> x = linspace(0,4*pi);  
>> a = sin(x);  
>> b = sin(2*x);  
>> c = sin(4*x);  
>> d = 7*a + 2*b + 3*c;  
>> plot(x,a,x,b,x,c,x,d)
```

Il comando **figure**; crea la finestra con un **nuovo grafico**.

Per **aggiungere una nuova funzione** su un grafico già aperto, si invia il comando **hold on**; prima della nuova istruzione **plot**.

